

Design and Evaluation of an FPGA-Based Context Encoder GAN for Embedded Image Inpainting

AKMAL HAFIZH MUSYAFFA¹ (Student Member, IEEE)

¹School of Electrical Engineering and Informatics, Institut Teknologi Bandung, Bandung, Indonesia (e-mail: akmal28hafizh@gmail.com)

Corresponding author: Akmal Hafizh Musyaffa (e-mail: akmal28hafizh@gmail.com).

ABSTRACT Image inpainting requires the reconstruction of missing image regions from surrounding visual context. The computational and memory requirements of deep generative models can limit their deployment on resource-constrained embedded platforms. This paper presents the design, verification, and FPGA implementation of a Context Encoder-based generative adversarial network (GAN) generator for image inpainting on the Digilent PYNQ-Z1 platform using a Xilinx Zynq-7000 XC7Z020 device. The accelerator implements a 12-layer encoder–bottleneck–decoder generator using signed 16-bit fixed-point arithmetic with eight fractional bits. The hardware architecture employs a unified convolution engine for standard and transposed convolutions, a 16-way pipelined multiply-accumulate datapath, ping-pong feature-map buffering, and external DDR memory accessed through an AXI4 master interface. The design is implemented using hand-written Verilog register-transfer-level logic. Post-implementation results show 7,393 lookup tables (13.90%), 3,244 flip-flops (3.05%), 50 DSP48E1 blocks (22.73%), and 16 RAMB18E1 primitives (5.71%) at a target frequency of 60 MHz. Layer-wise RTL verification against a PyTorch reference model is used to quantify numerical differences throughout the generator, with all layers satisfying the predefined MAE-based acceptance criteria and layer-wise correlation values remaining above 0.93. Hardware measurements report an accelerator latency of 5.55 s and an end-to-end latency of 5.60 s. Image quality is evaluated using RGB peak signal-to-noise ratio and structural similarity for both the complete image and the masked region.

INDEX TERMS context encoder, FPGA acceleration, fixed-point arithmetic, image inpainting, PYNQ-Z1

I. INTRODUCTION

IMAGE inpainting is the task of reconstructing missing, corrupted, or intentionally removed regions of an image. Unlike conventional image restoration, missing content is not directly observable and therefore must be inferred from the surrounding visual context. A successful reconstruction should preserve the spatial structure and appearance of the visible image while producing plausible content for the missing region.

Deep learning has enabled image-inpainting models to learn contextual representations directly from image data. Pathak *et al.* introduced the Context Encoder, an encoder–decoder formulation trained to predict missing image content from its surrounding context and investigated both reconstruction and adversarial objectives [1]. Subsequent methods introduced mechanisms to handle irregular missing regions.

Partial convolution conditions performed convolutional

operations on valid input pixels [2], while gated convolution introduced learned feature-wise gating for free-form image inpainting [3]. Although these methods improve reconstruction capability, their computational and memory requirements increase with model complexity and image resolution. This creates a practical challenge for embedded platforms with limited arithmetic resources, memory bandwidth, and power budgets.

Field-programmable gate arrays (FPGAs) provide configurable hardware datapaths that can exploit application-specific parallelism and reduced precision. Zynq-based systems are particularly relevant for embedded applications because the processing system (PS) and the programmable logic (PL) are integrated within a single device. Previous FPGA implementations have demonstrated convolutional neural-network (CNN) acceleration on Zynq platforms [4], [5], while Generative Adversarial Network (GAN) specific

accelerators have investigated the computational characteristics of generative workloads and transposed convolutions [7].

The implementation of an image-inpainting generator on a resource-constrained FPGA requires a balance between numerical precision, computational parallelism, memory traffic, and PL resources. This consideration is particularly relevant to the decoder, because transposed convolutions introduce data-access patterns that differ from conventional convolution.

This paper presents a hardware implementation of a Context Encoder-based image-inpainting generator targeting the Digilent PYNQ-Z1 platform and its Xilinx Zynq-7000 XC7Z020 programmable logic. The implementation uses signed 16-bit fixed-point arithmetic with eight fractional bits and is written entirely in hand-written Verilog register-transfer-level (RTL) logic. A unified convolution engine supports both standard and transposed convolutions, while intermediate feature maps are stored in external DDR memory using two ping-pong buffers. A 16-way pipelined multiply-accumulate (MAC) datapath provides the main arithmetic parallelism.

The main contributions of this work are as follows:

- A hand-written Verilog RTL implementation of a Context Encoder-based image-inpainting generator for the PYNQ-Z1/XC7Z020 platform.
- A unified convolution engine supporting both standard convolution and transposed convolution through a common datapath.
- An INT16 fixed-point implementation with eight fractional bits, together with layer-wise RTL verification against a PyTorch reference implementation.
- A hardware architecture combining a 16-way pipelined MAC datapath, on-chip weight storage, address generation, activation processing, and external DDR ping-pong feature-map buffers.
- An experimental evaluation covering FPGA resource utilization, timing closure, numerical error, inference latency, image quality, power, and energy per reconstructed image.

The remainder of this paper is organized as follows. Section II discusses image inpainting, the GAN formulation, and FPGA acceleration. Section III describes the generator architecture and fixed-point representation. Section IV presents the FPGA hardware architecture and RTL implementation. Section V describes the experimental methodology. Section VI presents the experimental results and discussion. Section VII concludes the paper.

II. RELATED WORK

A. DEEP LEARNING-BASED IMAGE INPAINTING

Traditional image-inpainting approaches commonly rely on diffusion, structural propagation, or patch-based synthesis. Such methods can perform effectively when suitable local structures are available, but large missing regions become difficult to reconstruct when the required semantic information is not present in the immediate neighborhood. Deep

generative approaches subsequently addressed this limitation by using learned image priors and semantic context to infer missing content [10], [11].

The Context Encoder introduced an encoder–decoder formulation in which the network learns a compact representation of the visible image context and reconstructs missing content [1]. Subsequent generative inpainting methods incorporated explicit contextual attention and multi-scale context modeling to improve structural and texture consistency [12], [13]. Later approaches introduced mechanisms specifically designed for irregular masks. Liu et al. proposed partial convolution, in which convolutional operations are conditioned on valid input pixels [2]. Yu et al. introduced gated convolution for free-form image inpainting, allowing the network to learn spatially varying feature selection during reconstruction [3].

These approaches illustrate a trade-off between reconstruction capability and computational requirements. More complex architectures are commonly designed for higher-resolution inputs and irregular masks, whereas an embedded implementation must account for available memory, arithmetic resources, and data-movement bandwidth. The present work therefore focuses on a compact 64×64 generator suitable for implementation on the XC7Z020.

B. CONTEXT ENCODER AND GAN-BASED ARCHITECTURES

The Context Encoder introduced by Pathak et al. uses an encoder–decoder structure to map visible image context into a compact latent representation and subsequently reconstruct the missing image content [1]. The original formulation combines reconstruction and adversarial objectives during training.

In the present work, the generator inference path is implemented in FPGA programmable logic, while the discriminator is not implemented as part of the hardware accelerator. The term “GAN” therefore refers to the training formulation of the generator rather than implying that the complete GAN training or discriminator inference is implemented on the FPGA.

C. GAN FORMULATION FOR CONTEXT-ENCODER INPAINTING

A generative adversarial network (GAN) consists of a generator G that produces samples and a discriminator D that distinguishes generated samples from real data. This generator–discriminator game is formulated as a two-player minimax optimization [8]. Convolutional GAN architectures subsequently demonstrated that adversarial training can be combined with convolutional generator and discriminator structures for image synthesis [9]. The conventional adversarial formulation is expressed as the two-player minimax objective

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]. \quad (1)$$

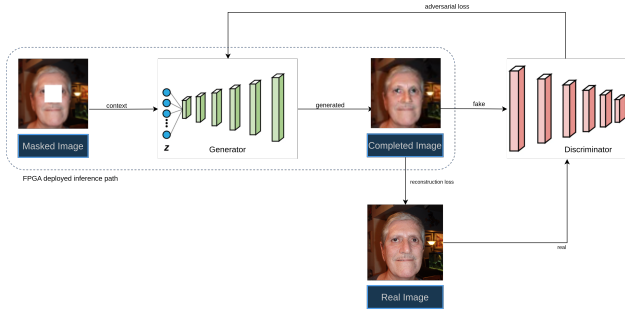


FIGURE 1. General GAN formulation adapted to context-encoder image inpainting. During training, the generator reconstructs the masked region and the discriminator distinguishes real and generated images. The FPGA implementation in this work deploys only the generator inference path.

For image inpainting, the generator is instead driven by the visible context of a masked image. Following the Context Encoder formulation, let $\hat{M}$ denote a binary mask whose value is one in the missing region and zero elsewhere. The masked input is therefore $(1 - \hat{M}) \odot x$. The reconstruction loss is evaluated only over the missing region:

$$L_{\text{rec}}(x) = \left\| \hat{M} \odot \left[x - F\left((1 - \hat{M}) \odot x\right) \right] \right\|_2^2, \quad (2)$$

where F denotes the context encoder generator and $\odot$ denotes element-wise multiplication. The corresponding adversarial objective used for the context encoder is

$$L_{\text{adv}} = \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{x \sim p_{\text{data}}} \left[\log \left(1 - D(F((1 - \hat{M}) \odot x)) \right) \right]. \quad (3)$$

The reconstruction and adversarial terms are combined as

$$L = \lambda_{\text{rec}} L_{\text{rec}} + \lambda_{\text{adv}} L_{\text{adv}}. \quad (4)$$

These equations describe the training formulation that motivates the Context Encoder generator. In the present implementation, the trained generator is deployed for inference in the FPGA programmable logic, whereas the discriminator and the training optimization procedure are not implemented in hardware.

The reconstruction-plus-adversarial formulation follows the original Context Encoder approach, in which the reconstruction term encourages structural consistency while the adversarial term encourages visually plausible completions [1].

D. FPGA ACCELERATION OF CNNs AND GANS

FPGAs have been investigated for neural-network acceleration because their programmable datapaths can exploit application-specific parallelism while supporting customized numerical representations. Prior accelerator studies emphasize that the balance between arithmetic throughput, on-chip storage, and external-memory bandwidth strongly influences

achievable performance [16], [17], [21]. Zynq-based implementations are particularly relevant to embedded systems because the processing system and programmable logic are integrated within a single system-on-chip.

ZynqNet demonstrated an embedded convolutional neural-network accelerator on a Xilinx Zynq platform and highlighted the importance of hardware-specific optimization for convolutional workloads [4]. CloudSatNet-1 investigated highly quantized CNN inference on a Zynq-7020 device [5]. Other accelerator studies have investigated 16-bit fixed-point computation and memory-oriented optimization for convolutional workloads. Low-precision fixed-point representations have also been studied as a means of reducing hardware cost while retaining useful numerical accuracy [14]. Data-movement-aware architectures such as Eyeriss further demonstrate the importance of reducing costly external-memory accesses [15]. [6]. GAN-specific FPGA acceleration has also been studied, including designs addressing the computational characteristics of transposed convolutions in generator networks [7].

Hand-coded RTL implementations have also been demonstrated for CNN acceleration, including FPGA-ARM systems that avoid dependence on HLS for the layer datapaths [19]. Other programmable FPGA flows have explored configurable CNN architectures and quantization strategies [18], while Caffeine and related approaches have investigated systematic compute and bandwidth optimization [20]. These studies provide architectural context for the present work, but their workloads, target devices, numerical formats, and optimization objectives differ from the compact Context Encoder-based image-inpainting generator considered here.

E. RESEARCH GAP

Existing image-inpainting research has largely emphasized reconstruction quality and increasingly complex network architectures, whereas FPGA research has addressed CNN classification and general neural-network acceleration. GAN-specific FPGA studies demonstrate the feasibility of accelerating generative workloads, but their target platforms and network structures differ from the architecture evaluated in this work.

The contribution of this work is therefore framed around the complete implementation and evaluation of a particular Context Encoder-based generator on a resource-constrained XC7Z020 device. The evaluation combines hand-written RTL implementation, fixed-point numerical verification, post-implementation resource analysis, hardware latency measurement, and separate assessment of the masked image region.

The objective is not to claim that no prior FPGA-based image-inpainting implementation exists. Instead, this study provides a reproducible hardware/software implementation and evaluation of the selected generator architecture on the PYNQ-Z1 platform.

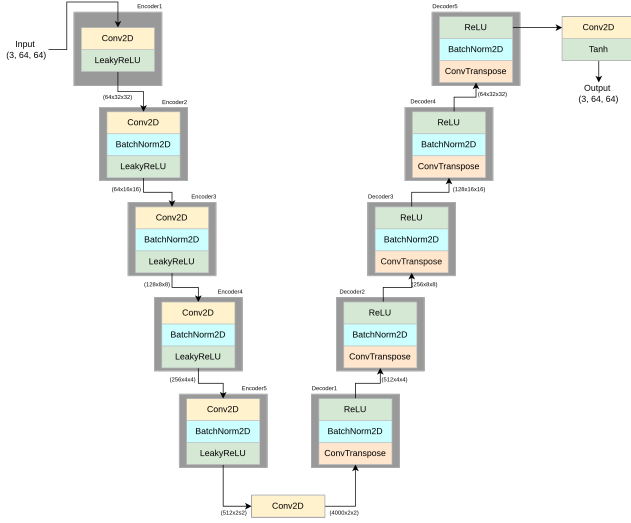


FIGURE 2. Sequential encoder-bottleneck-decoder organization of the Context Encoder-based generator. No skip connections are used.

III. PROPOSED CONTEXT ENCODER GAN ARCHITECTURE

A. NETWORK TOPOLOGY AND LAYER ORGANIZATION

The proposed generator operates on RGB images with an input dimension of $3 \times 64 \times 64$. A square missing region is introduced before inference, and the generator receives the resulting masked image as input. The network follows a sequential encoder-bottleneck-decoder topology without skip connections.

The encoder consists of five convolutional layers. Each layer reduces the spatial resolution by a factor of two while increasing the number of feature channels. A 1×1 bottleneck convolution processes the resulting 2×2 representation. The decoder consists of five transposed-convolution layers that restore the spatial resolution. Finally, a 3×3 convolution followed by a Tanh activation produces the reconstructed RGB image.

Table 1 summarizes the structural and computational configuration of the generator.

The generator contains 5,856,451 trainable parameters and approximately 241.04 million multiply-accumulate operations (MACs) according to the computational accounting used in this work. The reported parameter total includes the trainable parameters associated with the convolutional and normalization layers.

The MAC count is reported using the same layer-wise accounting adopted for the implementation analysis, including the transposed-convolution layers. Counting one multiplication and one addition as two floating point operations gives approximately 482.08 MFLOPs.

B. FIXED-POINT REPRESENTATION

The FPGA implementation uses signed 16-bit fixed-point values with eight fractional bits. The scale factor is

$$S = 2^8 = 256. \quad (5)$$

The input image is normalized using

$$x_{\text{norm}} = \frac{x_{\text{pixel}}}{127.5} - 1. \quad (6)$$

The normalized value is quantized as

$$x_{\text{int16}} = \text{clip}(\text{round}(x_{\text{norm}} \times 256), -32768, 32767). \quad (7)$$

For consistency throughout the manuscript, the numerical format is described as signed INT16 with eight fractional bits. The terms Q7.8 or Q8.8 are avoided unless the exact signed-range convention is explicitly defined.

For two operands represented with eight fractional bits, multiplication produces a value with sixteen fractional bits. The product is accumulated using a signed 48-bit accumulator. After accumulation, the result is rescaled to the eight-fractional-bit representation and saturated to the signed 16-bit range.

$$y_{\text{int16}} = \text{sat}_{16} \left(\frac{\sum_i x_i w_i + b}{2^8} \right). \quad (8)$$

The output tensor is subsequently dequantized by dividing the integer values by 256 before conversion to the image representation.

C. NORMALIZATION PARAMETERS

The trained model contains normalization operations in the generator architecture. During hardware implementation, the corresponding parameters are represented in fixed-point form and supplied to the RTL datapath.

The exact treatment of normalization parameters must remain consistent between the PyTorch reference model, parameter-extraction script, RTL implementation, and FPGA execution. In particular, if normalization parameters are folded into convolution parameters in the final implementation, the folding procedure should be explicitly described and the resulting parameter count should be reported accordingly.

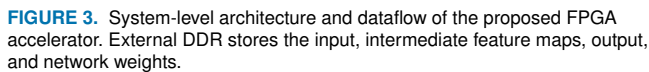
IV. FPGA HARDWARE ARCHITECTURE AND IMPLEMENTATION

A. SYSTEM-LEVEL TARGET PLATFORM

The accelerator is implemented on the Digilent PYNQ-Z1 platform using the Xilinx Zynq-7000 XC7Z020 device. The system consists of an ARM-based processing system (PS) and programmable logic (PL). The PS executes the PYNQ software environment and controls the accelerator, while the PL implements the generator datapath.

Communication between the PS and accelerator is divided into control and data paths. An AXI4-Lite interface is used for configuration and status registers, while the accelerator accesses external DDR memory through an AXI4 master interface.

	Layer	Type	Input	Output	Kernel	Stride	Pad.	Activation	Parameters	MACs
	E1	Conv2D	3×64^2	32×32^2	4×4	2	1	LeakyReLU (0.2)	1,568	1.57M
	E2	Conv2D	32×32^2	64×16^2	4×4	2	1	LeakyReLU (0.2)	32,832	8.39M
	E3	Conv2D	64×16^2	128×8^2	4×4	2	1	LeakyReLU (0.2)	131,456	8.39M
	E4	Conv2D	128×8^2	256×4^2	4×4	2	1	LeakyReLU (0.2)	525,056	8.39M
	E5	Conv2D	256×4^2	512×2^2	4×4	2	1	LeakyReLU (0.2)	2,098,688	8.39M
	BOT	Conv2D	512×2^2	512×2^2	1×1	1	0	None	262,656	1.05M
	D1	ConvTranspose2D	512×2^2	256×4^2	4×4	2	1	ReLU	2,097,920	33.55M
	D2	ConvTranspose2D	256×4^2	128×8^2	4×4	2	1	ReLU	524,672	33.55M
	D3	ConvTranspose2D	128×8^2	64×16^2	4×4	2	1	ReLU	131,264	33.55M
	D4	ConvTranspose2D	64×16^2	32×32^2	4×4	2	1	ReLU	32,864	33.55M
	D5	ConvTranspose2D	32×32^2	32×64^2	4×4	2	1	ReLU	16,480	67.11M
	OUT	Conv2D	32×64^2	3×64^2	3×3	1	1	Tanh	867	3.54M
								Total	5,856,451	$\approx 241.04\text{M}$



B. RTL MODULE ORGANIZATION

The `gan_axi_wrapper` provides the interface between the Zynq processing system and the generator core. It handles the memory-mapped control registers and exposes accelerator status to the software environment.

The `generator_top` module provides top-level execution control. An internal finite-state machine sequences the twelve generator layers. The active layer is advanced sequentially from E1 through OUT.

The `conv_unified` module provides a common datapath for standard and transposed convolution. Layer-specific configuration determines the convolution mode, tensor dimensions, stride, padding, activation function, and memory locations. Its functional microarchitecture is shown in Fig. 4. The address-generation path computes the spatial

The MAC datapath receives the selected input values and cached weights. Sixteen signed INT16 multipliers operate in parallel, and their products are sign-extended before reduction by the pipelined adder tree. The resulting partial sum is passed through the accumulator, where the layer bias is selected and the accumulated value is retained across the required input-channel contributions. The fixed-point output path then performs rounding, right shifting to restore the eight-fractional-bit representation, and signed 16-bit saturation. Finally, a layer-controlled activation path selects LeakyReLU, ReLU, Tanh, or no activation before the result is written to the destination feature-map buffer.

The figure represents the functional datapath inside `conv_unified` and its associated local storage and pipeline registers; individual blocks in the figure do not necessarily correspond to separate Verilog modules.

The `mac_array` contains 16 parallel signed INT16 multiplication lanes followed by a two-stage pipeline. The first stage registers multiplication results, while the second stage performs an adder-tree reduction. Partial sums are accumulated using a signed 48-bit representation.

The `addr_gen` module generates feature-map and weight addresses from the active spatial position, channel indices, layer configuration, and convolution mode.

Activation modules implement LeakyReLU, ReLU, and Tanh operations. Bias values are loaded through the control interface and stored locally for use during layer execution.

External DDR memory is used as the primary storage medium for network weights and intermediate feature maps. The principal runtime buffers are:

- input image buffer,
- feature-map buffer A,
- feature-map buffer B,

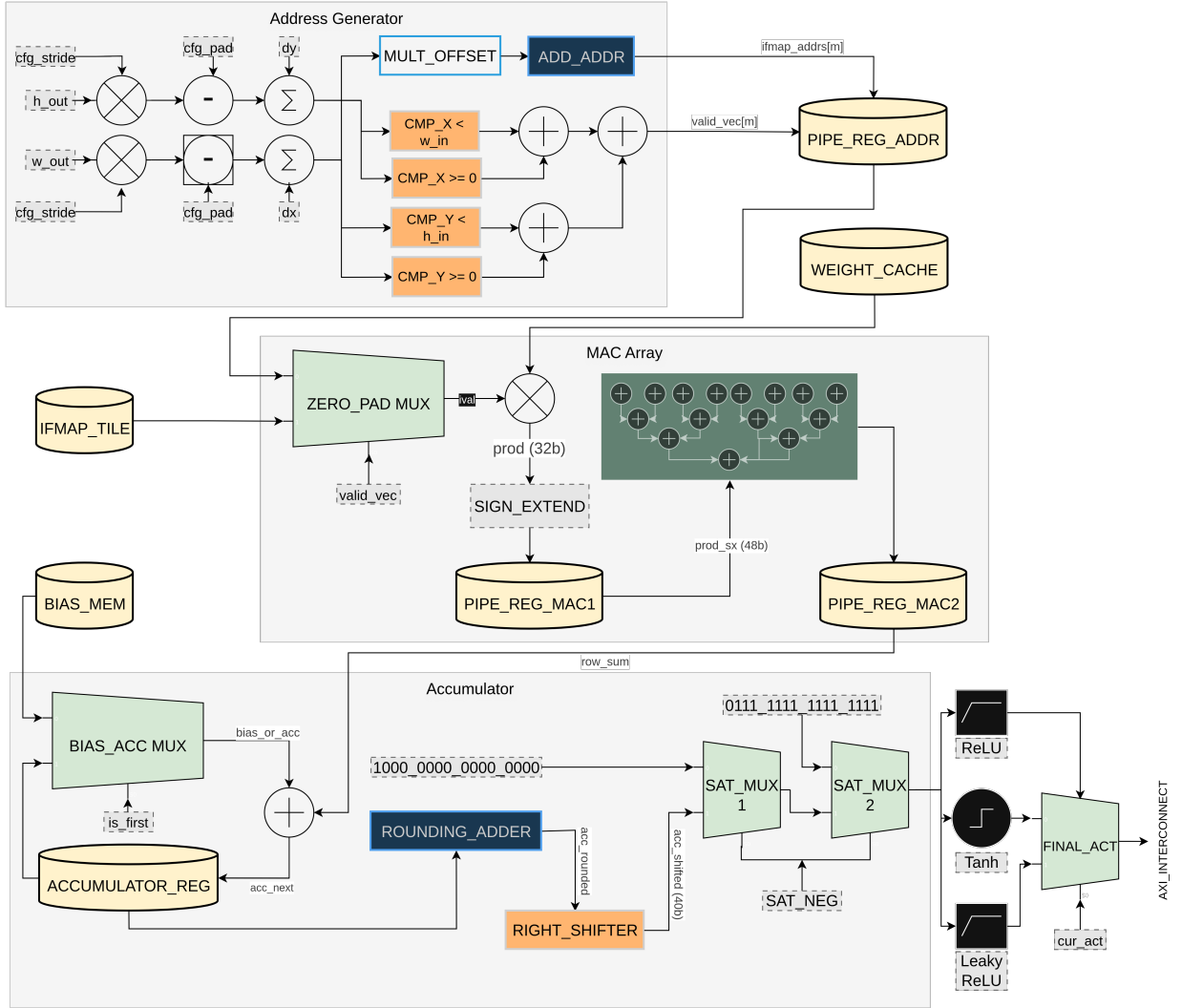


FIGURE 4. Detailed functional datapath of the unified convolution engine. The address generator forms input-feature-map addresses and tap-validity information, the zero-padding multiplexer selects valid input values, the 16-way MAC array performs parallel signed INT16 multiplications and pipelined reduction, and the accumulator performs bias addition, rounding, fixed-point rescaling, saturation, and layer-dependent activation.

- output image buffer, and
- weight buffer.

Intermediate feature maps alternate between buffers A and B after each layer. This avoids an explicit feature-map copy between consecutive layers because the destination buffer of one layer becomes the source buffer of the following layer.

The network weights are stored in external DDR and accessed through the AXI4 master interface. The convolution engine also contains small on-chip weight-cache banks to reduce repeated external-memory accesses.

The resulting architecture is primarily DDR-centric. External memory provides the storage capacity required by the generator, while the programmable logic contains the control logic, address generation, arithmetic datapath, activation functions, and local storage.

TABLE 2. Principal AXI4-Lite Control Registers

Offset	Function
0x00	Start control
0x04	Status
0x08	Input image base address
0x0C	Feature-map A base address
0x10	Feature-map B base address
0x14	Output base address
0x18	Bias configuration
0x1C	Bias data
0x20	Weight base address

D. AXI CONTROL AND REGISTER INTERFACE

The accelerator is controlled using an AXI4-Lite memory-mapped register interface. The principal registers are summarized in Table 2.

The status register exposes the accelerator completion

state, current layer index, and FSM state. The software starts an inference by writing to the start register and polls the status register until completion.

The buffer addresses are allocated dynamically at runtime. Therefore, the addresses used by the PYNQ software are not treated as fixed physical addresses in the manuscript.

V. EXPERIMENTAL METHODOLOGY

A. DATASET AND INPUT PREPROCESSING

The generator is evaluated using the FFHQ dataset at a resolution of 64×64 . Images are resized using bicubic interpolation before normalization and fixed-point quantization.

The normalized input is represented using signed INT16 arithmetic with eight fractional bits. A square missing region is then introduced according to the selected mask configuration.

For center-mask experiments, the evaluated mask sizes are 10, 16, 24, and 32 pixels. Random masks are additionally considered to evaluate reconstruction under less structured missing regions.

A random test set of 1,000 images is used for the quantitative evaluation. The reported image-quality metrics are computed over this test set for each mask configuration.

B. SOFTWARE REFERENCE AND WEIGHT CONVERSION

A PyTorch implementation is used as the software reference model. The trained generator parameters are extracted and converted into signed fixed-point hexadecimal representations for RTL simulation and FPGA execution.

The conversion pipeline preserves the ordering of the generator parameters and generates the parameter files consumed by the RTL implementation. The PyTorch reference implementation is also used to generate intermediate feature-map tensors for numerical comparison.

C. RTL VERIFICATION

Functional verification is performed using Verilator. The simulation testbench models external DDR memory, loads the input image and network parameters, drives the AXI interfaces, and records intermediate feature maps produced by the RTL accelerator.

Each generated intermediate tensor is compared with the corresponding PyTorch reference tensor. The comparison uses maximum absolute error, mean absolute error (MAE), correlation, and predefined tolerance values.

The verification status reported in this work is based on the MAE criterion. Exact INT16 equality is not assumed. Therefore, the results are described as numerical agreement within predefined tolerances rather than as bit-exact equivalence.

D. FPGA IMPLEMENTATION AND TIMING

The RTL design is synthesized, placed, and routed using Xilinx Vivado 2023.2 for the XC7Z020 device. The target programmable-logic clock frequency is 60 MHz, corresponding to a clock period of approximately 16.667 ns.

Timing closure is evaluated using the post-implementation Vivado timing report, while resource utilization is obtained from the post-implementation utilization report.

The Verilator simulation requires 326,774,321 clock cycles for one complete generator execution. At a 60 MHz clock frequency, the corresponding cycle-derived execution time is

$$T_{\text{cycle}} = \frac{326,774,321}{60 \times 10^6} \approx 5.44 \text{ s.} \quad (9)$$

This value is treated as a cycle-derived estimate. It is reported separately from the measured execution time on the physical PYNQ-Z1 platform.

E. IMAGE QUALITY EVALUATION

Image reconstruction quality is evaluated against the original unmasked image using peak signal-to-noise ratio (PSNR) and structural similarity index (SSIM).

Because pixels outside the missing region can be restored directly from the original image during final compositing, full-image metrics can overestimate the quality of the generated content. Therefore, both the complete reconstructed image and the masked region should be evaluated.

PSNR is calculated as

$$\text{PSNR} = 10 \log_{10} \left(\frac{MAX_I^2}{\text{MSE}} \right), \quad (10)$$

where MAX_I denotes the maximum possible image intensity and MSE is calculated between the generated and ground-truth pixels.

SSIM is calculated using the standard structural-similarity formulation. The evaluation is performed on RGB images rather than converting the images to grayscale.

F. POST-PROCESSING

The raw generator output is dequantized from the fixed-point representation before conversion to image format. During final reconstruction, pixels outside the masked region are restored from the original image.

The implementation also contains a mean-color correction procedure based on pixels surrounding the masked region. However, the primary image-quality results reported in this work are obtained without mean-color correction. This configuration uses the raw dequantized hardware output to isolate the effect of the fixed-point generator. Since this operation is performed after generator inference, it is treated as post-processing rather than part of the FPGA generator datapath.

G. LATENCY, POWER, AND ENERGY MEASUREMENT

Hardware latency is measured on the PYNQ-Z1 rather than inferred solely from simulation time. Latency statistics are obtained from 100 repeated inference runs on the PYNQ-Z1. The mean, standard deviation, minimum, and maximum latencies are reported for both the embedded ARM processor

TABLE 3. Post-Implementation FPGA Resource Utilization

Resource	Used	Available	Utilization
LUT	7,393	53,200	13.90%
Flip-Flop	3,244	106,400	3.05%
RAMB18E1	16	280	5.71%
DSP48E1	50	220	22.73%

and the programmable-logic accelerator. The same repeated-run procedure is used to characterize the latency statistics reported in Table 8. Two latency measures are distinguished:

- 1) **Accelerator latency**, measured from hardware start until the accelerator completion status is asserted.
- 2) **End-to-end latency**, including software-controlled preprocessing, buffer preparation, accelerator execution, and post-processing where applicable.

Power measurements distinguish idle and active operation:

$$P_{\text{dynamic}} = P_{\text{active}} - P_{\text{idle}}. \quad (11)$$

Energy per image is calculated as

$$E_{\text{image}} = P_{\text{active}} T_{\text{inference}}. \quad (12)$$

VI. RESULTS AND DISCUSSION

A. FPGA RESOURCE UTILIZATION

The post-implementation Vivado report gives the resource utilization shown in Table 3.

The design uses 7,393 LUTs and 3,244 flip-flops, corresponding to 13.90% and 3.05% of the available resources, respectively. DSP48E1 utilization is 50 blocks (22.73%), while 16 RAMB18E1 primitives (5.71%) are used.

The relatively small BRAM utilization is consistent with the use of external DDR memory for intermediate feature maps and network weights. The implementation therefore does not require all intermediate tensors to reside in on-chip block RAM.

Vivado reports eight block-RAM tiles corresponding to 16 RAMB18E1 primitives. The primitive count is used in Table 3.

B. TIMING CLOSURE

The design uses a target clock frequency of 60 MHz, corresponding to a period of approximately 16.667 ns. Post-implementation timing analysis reports timing closure at this operating point.

The 60 MHz frequency represents the selected operating target and should not be interpreted as the maximum achievable frequency of the XC7Z020.

C. LAYER-WISE NUMERICAL VERIFICATION

The RTL simulation compares intermediate tensors against the PyTorch reference after each generator layer. This layer-wise procedure allows numerical differences to be examined before they propagate to subsequent layers.

TABLE 4. Layer-Wise Numerical Verification

Layer	Status	Max Error	MAE	Correlation	Tolerance
E1	PASS	0.01953	0.00230	1.0000	0.01
E2	PASS	0.13672	0.01083	0.9993	0.02
E3	PASS	0.19141	0.02045	0.9987	0.03
E4	PASS	0.20312	0.02765	0.9977	0.05
E5	PASS	0.28125	0.04081	0.9971	0.08
BOT	PASS	1.95312	0.44825	0.9963	0.60
D1	PASS	0.96875	0.08613	0.9675	0.15
D2	PASS	0.73438	0.05611	0.9808	0.10
D3	PASS	0.70312	0.03665	0.9877	0.08
D4	PASS	0.50000	0.01822	0.9863	0.05
D5	PASS	3.23047	0.04994	0.9341	0.08
OUT	PASS	1.73438	0.17040	0.9419	0.20

TABLE 5. Inference Latency and Throughput

Metric	Value
Clock frequency	60 MHz
Simulation cycle count	326,774,321
Cycle-derived latency	~5.44 s
Measured accelerator latency	5.55 s
End-to-end latency	5.60 s
Throughput	0.178 FPS

Table 4 summarizes the measured errors.

The encoder and bottleneck layers exhibit correlations above 0.99. Lower correlations are observed in several decoder layers, with the lowest value occurring at D5. The largest maximum absolute error is 3.23047 at D5, while the corresponding MAE remains below the specified acceptance threshold.

The MAE increases from 0.00230 at E1 to 0.44825 at the bottleneck before remaining below 0.171 in the final decoder and output layers, while the correlation remains above 0.93 throughout the generator.

The PASS classification is based on MAE rather than maximum absolute error. This distinction is important for E1, where the maximum absolute error is larger than the listed tolerance although the MAE satisfies the acceptance criterion.

The decrease in correlation in some decoder layers does not by itself establish a specific source of error. It is consistent with the accumulation of numerical differences through successive fixed-point operations, but additional error-propagation analysis would be required to attribute the differences to a particular layer or operation.

D. INFERENCE LATENCY AND THROUGHPUT

The complete generator execution requires 326,774,321 clock cycles in the Verilator simulation. At 60 MHz, this corresponds to approximately 5.44 s of cycle-derived execution time.

Table 5 summarizes the cycle-derived and measured execution results.

The layer-wise cycle distribution shows that the decoder dominates the execution time. D5 requires the largest number

TABLE 6. Layer-Wise RTL Execution Cycles

Layer	RTL Cycles
E1	4,065,826
E2	20,568,068
E3	19,652,612
E4	18,202,628
E5	16,654,340
BOT	11,804,676
D1	33,964,036
D2	36,044,804
D3	37,773,316
D4	38,852,612
D5	79,064,068
OUT	10,127,335

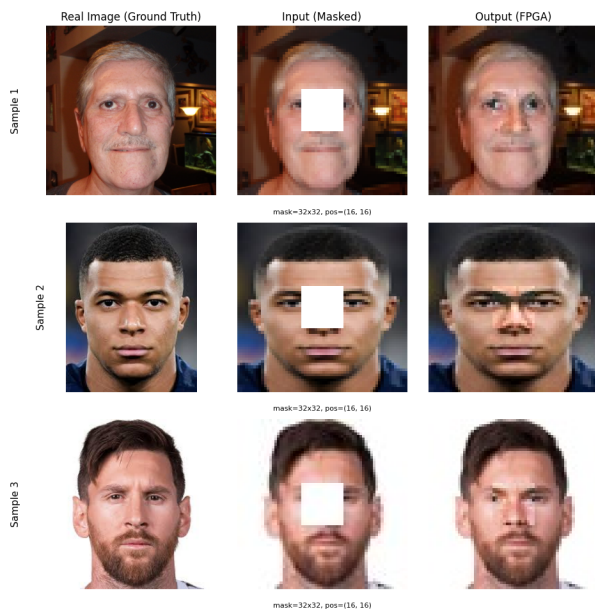


FIGURE 5. Qualitative comparison of the original image, masked input, and reconstructed output.

of cycles at 79,064,068 cycles, accounting for approximately 24.2% of the total generator cycle count. This behavior is associated with the larger spatial dimensions processed in the later decoder stages.

The measured accelerator latency is 5.55 s, while the measured end-to-end latency is 5.60 s. The difference between these measurements represents the software and system overhead included in the end-to-end measurement.

The cycle-derived latency and hardware measurement are reported separately because the former is obtained from RTL simulation whereas the latter is measured during execution on the physical PYNQ-Z1 platform.

E. IMAGE RECONSTRUCTION QUALITY

The reconstructed images are evaluated using RGB PSNR and SSIM.

The image-quality measurements are summarized in Table 7, including both full-image and masked-region evaluation.

The mask-region metrics consistently remain lower than

TABLE 7. Image-Inpainting Quality Under Different Mask Conditions

Mask Condition	PSNR Full	SSIM Full	PSNR Mask	SSIM Mask
Center, 10 px	28.52	0.942	23.41	0.810
Center, 16 px	26.15	0.895	21.05	0.745
Center, 24 px	22.80	0.821	18.22	0.612
Center, 32 px	19.45	0.704	15.30	0.521
Random	24.10	0.850	19.75	0.680

TABLE 8. CPU and FPGA Performance Comparison

Metric	ARM CPU	PYNQ-Z1 PL
Precision	FP32	INT16
Average latency	29.50 s	5.60 s
Std. deviation	0.85 s	0.012 s
Minimum latency	28.42 s	5.58 s
Maximum latency	31.15 s	5.63 s
Throughput	0.034 FPS	0.178 FPS

the corresponding full-image metrics, indicating that full-image evaluation benefits from the preserved unmasked pixels. As the center-mask size increases, both full-image and mask-region PSNR and SSIM decrease, reflecting the increasing amount of content that must be reconstructed.

The full-image metrics decrease as the center-mask size increases. This trend is consistent with the increasing amount of content that must be inferred by the generator.

However, the full-image values are affected by the fact that pixels outside the missing region can be restored directly from the original image during final compositing. Consequently, these values should not be interpreted as an isolated measure of generated-region quality.

F. CPU-FPGA COMPARISON

The CPU comparison distinguishes between a host-machine PyTorch measurement and the embedded ARM processor on the PYNQ-Z1.

The host-machine FP32 PyTorch implementation requires approximately 471.26 ms per inference in the reported software measurement. This result is not used as the embedded CPU baseline because it was obtained on a different computing platform.

For the embedded comparison, the generator is executed on the ARM Cortex-A9 processor integrated in the PYNQ-Z1.

Under the reported benchmark conditions, the programmable logic implementation has lower measured latency than the embedded ARM CPU baseline. The two implementations use different numerical representations, with FP32 used for the CPU baseline and INT16 fixed-point arithmetic used by the FPGA accelerator. Therefore, this comparison is interpreted as a platform-level latency comparison rather than an isolated comparison of arithmetic efficiency.

The corresponding latency ratio is approximately

$$\text{Accelerator Speedup} = \frac{29.50}{5.55} \approx 5.32 \times . \quad (13)$$

TABLE 9. Power and Energy Measurement

Metric	Measured Value
Idle power	1.50 W
Active power	2.45 W
Dynamic difference	0.95 W
Accelerator latency	5.55 s
Energy/image	13.59 J

This ratio compares the embedded ARM inference latency with the measured accelerator latency and excludes the small software overhead included in the end-to-end measurement. The corresponding ratio based on the end-to-end latency is approximately $29.50/5.60 \approx 5.27\times$. This ratio should be described as a measured latency reduction relative to the embedded ARM baseline under the stated experimental conditions, rather than as a general FPGA speedup.

G. POWER AND ENERGY

Power consumption is evaluated under idle and active inference conditions.

The measured active power is 2.45 W, while the difference between the active and idle measurements is 0.95 W. Using the measured active power of 2.45 W and accelerator latency of 5.55 s, the resulting energy consumption is approximately

$$E_{\text{image}} = 2.45 \times 5.55 \approx 13.59 \text{ J/image.} \quad (14)$$

The reported active and idle power values correspond to the total PYNQ-Z1 board power measured during the experiment. These values describe the measured operating condition of the platform and are not interpreted as an efficiency advantage without a directly comparable power measurement obtained under equivalent conditions.

H. COMPARISON WITH PRIOR FPGA ACCELERATORS

Table 10 provides architectural context by listing representative FPGA-based neural-network implementations. Because the workloads, devices, numerical formats, and measurement procedures differ, the values are not treated as directly comparable benchmarks.

The comparison is intended to provide architectural context rather than establish direct superiority. The proposed implementation targets a different workload and emphasizes the deployment of a Context Encoder-based image-inpainting generator on the XC7Z020 platform.

I. TRADE-OFFS AND LIMITATIONS

The implementation involves several hardware trade-offs.

First, INT16 arithmetic reduces the arithmetic width relative to FP32 computation while retaining more numerical precision than very low-bit representations. The numerical verification results nevertheless show that differences between the hardware and software representations become more visible in several decoder layers. The lowest layer-wise correlation is observed at D5, at 0.9341, while the output

layer retains a correlation of 0.9419. These results indicate that the fixed-point implementation maintains overall numerical agreement while exhibiting increasing deviation in the later decoder stages.

Second, the use of external DDR reduces the amount of on-chip BRAM required for feature-map storage but increases dependence on external memory accesses. The architecture therefore trades on-chip storage requirements for memory traffic.

Third, the 16-way MAC datapath provides a fixed level of arithmetic parallelism. Increasing the number of parallel lanes could reduce inference latency, but would also increase DSP utilization, routing complexity, and memory-bandwidth requirements.

Finally, the current implementation operates at 64×64 resolution. Increasing the input resolution would increase both the number of convolution operations and the amount of intermediate feature-map data transferred through external memory.

VII. CONCLUSION

This paper presented a hand-written Verilog RTL implementation of a Context Encoder-based image-inpainting generator on the PYNQ-Z1 platform using the Xilinx Zynq-7000 XC7Z020 device. The generator consists of five encoder layers, a bottleneck layer, five decoder layers, and an output convolution layer, operating on 64×64 RGB images.

The hardware implementation uses a unified convolution engine for standard and transposed convolutions, a 16-way pipelined MAC datapath, local weight storage, address generation, activation processing, and external DDR ping-pong buffers. The numerical representation uses signed INT16 fixed-point values with eight fractional bits.

Post-implementation results show utilization of 7,393 LUTs (13.90%), 3,244 flip-flops (3.05%), 50 DSP48E1 blocks (22.73%), and 16 RAMB18E1 primitives (5.71%). Timing closure is achieved at the selected 60 MHz operating frequency.

Layer-wise RTL verification against the PyTorch reference model shows numerical agreement under the predefined MAE-based acceptance criteria. The measured accelerator latency on the PYNQ-Z1 is 5.55 s, with an end-to-end latency of 5.60 s. The embedded ARM baseline requires approximately 29.50 s under the reported experimental conditions, resulting in a measured latency ratio of approximately $5.32\times$.

The measured active power is 2.45 W. Using the accelerator latency gives an estimated energy consumption of approximately 13.59 J per image under the stated measurement condition.

Image-quality evaluation shows a decrease in full-image PSNR and SSIM as the masked-region size increases. For center masks, full-image PSNR decreases from 28.52 dB for the 10-pixel mask to 19.45 dB for the 32-pixel mask, while the corresponding masked-region PSNR decreases from 23.41 dB to 15.30 dB. However, because unmasked pixels are restored directly from the original image during

TABLE 10. Comparison With Representative FPGA-Based Neural-Network Accelerators

Work	Task	Device	Prec.	Freq.	LUT	FF	BRAM	DSP	Performance
ZynqNet [4]	CNN classification	XC7Z045	FP32	200 MHz	~70%	~31%	~91%	~82%	0.51 FPS
CloudSatNet-1 [5]	CNN classification	Zynq-7020	4-bit	N/R	66.67%	48.14%	61.43%	13.64%	9.93 FPS
FlexiGAN [7]	GAN	XCVU13P	INT16	~190 MHz	77%	N/R	93%	13%	2.2×
Proposed	Inpainting	XC7Z020	INT16	60 MHz	13.90%	3.05%	5.71%	22.73%	0.178 FPS

compositing, mask-region-only metrics are required to isolate the actual quality of the generated content.

The current implementation is limited to 64×64 images and uses external DDR for most intermediate data storage. Future work can therefore investigate memory-traffic reduction, increased arithmetic parallelism, improved throughput, and extension toward higher resolutions and more irregular mask configurations.

ACKNOWLEDGMENT

The authors acknowledge the use of AI-assisted tools for language editing and manuscript organization. All technical content, numerical results, experimental procedures, interpretations, and final revisions were reviewed and verified by the authors.

REFERENCES

- [1] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros, "Context encoders: Feature learning by inpainting," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Jun. 2016, pp. 2536–2544.
- [2] G. Liu, F. A. Reda, K. J. Shih, T.-C. Wang, A. Tao, and B. Catanzaro, "Image inpainting for irregular holes using partial convolutions," in Proc. Eur. Conf. Comput. Vis. (ECCV), 2018, pp. 85–100.
- [3] J. Yu, Z. Lin, J. Yang, X. Shen, X. Lu, and T. S. Huang, "Free-form image inpainting with gated convolution," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 4471–4480.
- [4] D. Gschwend, "ZynqNet: An FPGA-accelerated embedded convolutional neural network," M.S. thesis, Dept. Inf. Technol. Elect. Eng., ETH Zurich, Zurich, Switzerland, 2016.
- [5] R. Pitonak, J. Mucha, L. Dobis, M. Javorka, and M. Marusin, "CloudSatNet-1: FPGA-based hardware-accelerated quantized CNN for satellite on-board cloud coverage classification," Remote Sens., vol. 14, no. 13, p. 3180, 2022.
- [6] A. Alhussain and M. Lin, "Hardware-efficient template-based deep CNNs accelerator design," in Proc. IEEE Int. Conf. Networking, Architecture and Storage (NAS), 2022, pp. 1–4.
- [7] A. Yazdanbakhsh, M. Brzozowski, B. Khaleghi, S. Ghodrati, K. Samadi, N. S. Kim, and H. Esmailzadeh, "FlexiGAN: An end-to-end solution for FPGA acceleration of generative adversarial networks," in Proc. IEEE Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM), 2018, pp. 65–72.
- [8] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in Advances in Neural Information Processing Systems (NeurIPS), vol. 27, 2014, pp. 2672–2680.
- [9] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in Proc. Int. Conf. Learn. Represent. (ICLR), 2016.
- [10] R. A. Yeh, C. Chen, T. Y. Lim, A. G. Schwing, M. Hasegawa-Johnson, and M. N. Do, "Semantic image inpainting with deep generative models," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 5485–5493.
- [11] S. Iizuka, E. Simo-Serra, and H. Ishikawa, "Globally and locally consistent image completion," ACM Trans. Graph., vol. 36, no. 4, 2017, Art. no. 107, pp. 1–14.
- [12] J. Yu, Z. Lin, J. Yang, X. Shen, X. Lu, and T. S. Huang, "Generative image inpainting with contextual attention," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 5505–5514.
- [13] Y. Zeng, J. Fu, H. Chao, and B. Guo, "Learning pyramid-context encoder network for high-quality image inpainting," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2019, pp. 1486–1494.
- [14] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in Proc. 32nd Int. Conf. Mach. Learn. (ICML), 2015, pp. 1737–1746.
- [15] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," IEEE J. Solid-State Circuits, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [16] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA), 2015, pp. 161–170.
- [17] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song, Y. Wang, and H. Yang, "Going deeper with embedded FPGA platform for convolutional neural network," in Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA), 2016, pp. 26–35.
- [18] K. Guo, L. Sui, J. Qiu, J. Yu, J. Wang, S. Yao, S. Han, Y. Wang, and H. Yang, "Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 37, no. 1, pp. 35–47, Jan. 2018.
- [19] R. Ding, G. Su, G. Bai, W. Xu, N. Su, and X. Wu, "A FPGA-based accelerator of convolutional neural network for face feature extraction," in Proc. IEEE Int. Conf. Electron Devices Solid-State Circuits (EDSSC), 2019, pp. 1–3, doi: 10.1109/EDSSC.2019.8754067.
- [20] C. Zhang, G. Sun, Z. Fang, P. Zhou, P. Pan, and J. Cong, "Caffeine: Toward uniformed representation and acceleration for deep convolutional neural networks," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 38, no. 11, pp. 2072–2085, Nov. 2019.
- [21] S. I. Venieris, A. Kouris, and C.-S. Bouganis, "Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions," ACM Comput. Surv., vol. 51, no. 3, Art. no. 56, pp. 1–39, 2018, doi: 10.1145/3186332.